The slot machine example may not seem to be a particularly real-world problem, but if we add one element, it does become a practical business problem, with one big example being advertisement placement. Whenever you visit a website with ads, the company placing the ads wants to maximize the probability that you will click them.

Let's say we manage 10 e-commerce websites, each focusing on selling a different broad category of retail items such as computers, shoes, jewelry, etc. We want to increase sales by referring customers who shop on one of our sites to another site that they might be interested in. When a customer checks out on a particular site in our network, we will display an advertisement to one of our other sites in hopes they'll go there and buy something else. Alternatively, we could place an ad for another product on the same site. Our problem is that we don't know which sites we should refer users to. We could try placing random ads, but we suspect a more targeted approach is possible.

2.3.1. Contextual bandits

Perhaps you can see how this just adds a new layer of complexity to the *n*-armed bandit problem we considered at the beginning of the chapter. At each play of the game (each time a customer checks out on a particular website) we have $n = 10$ possible actions we can take, corresponding to the 10 different types of advertisements we could place. The twist is that the best ad to place may depend on which site in the network the current customer is on. For example, a customer checking out on our jewelry site may be more in the mood to buy a new pair of shoes to go with their new diamond necklace than they would be to buy a new laptop. Thus our problem is to figure out how a particular site relates to a particular advertisement.

This leads us to *state spaces*. The *n*-armed bandit problem we started with had an *n*-element *action space* (the space or set of all possible actions), but there was no concept of state. That is, there was no information in the environment that would help us choose a good arm. The only way we could figure out which arms were good is by trial and error. In the ad problem, we know the user is buying something on a particular site, which may give us